

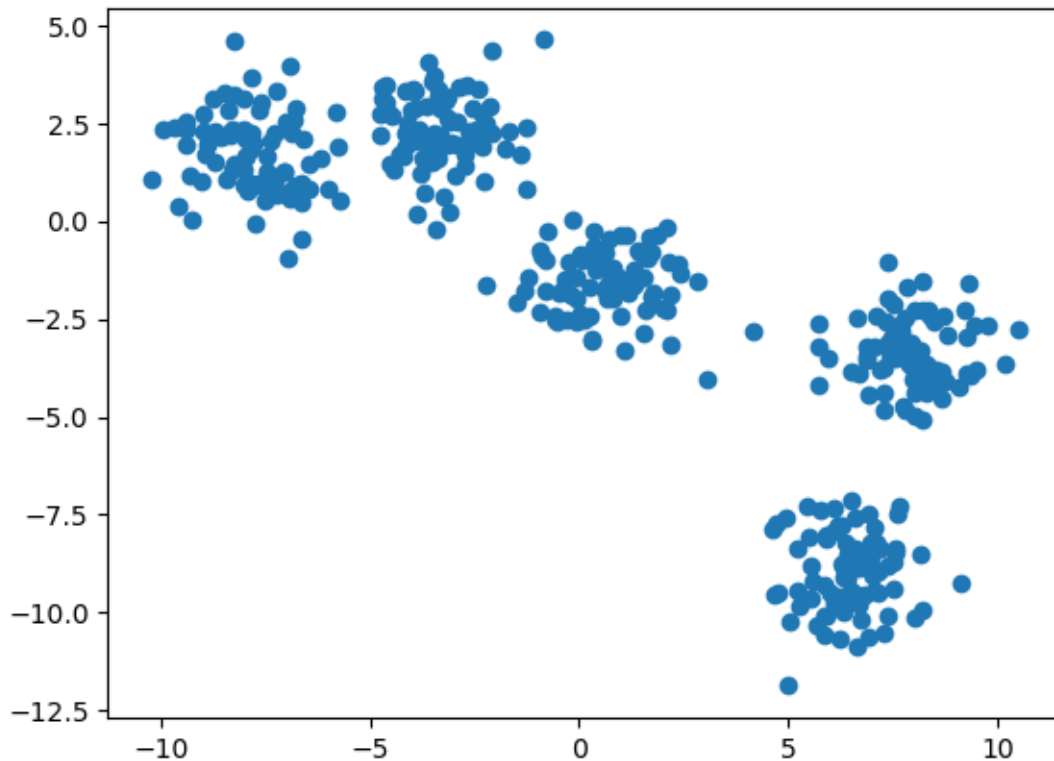
Kmeans

Guillermo Ruiz

En este notebook se muestra la diferencia al elegir el número de clusters para un conjunto de datos. Los datos los vamos a crear con la función `make_blobs` que crea clusters de manera artificial, en este caso son 5 clusters.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

```
X, y = make_blobs(400, 2, centers=5, random_state=6)
plt.scatter(X[:, 0], X[:, 1])
plt.show()
```

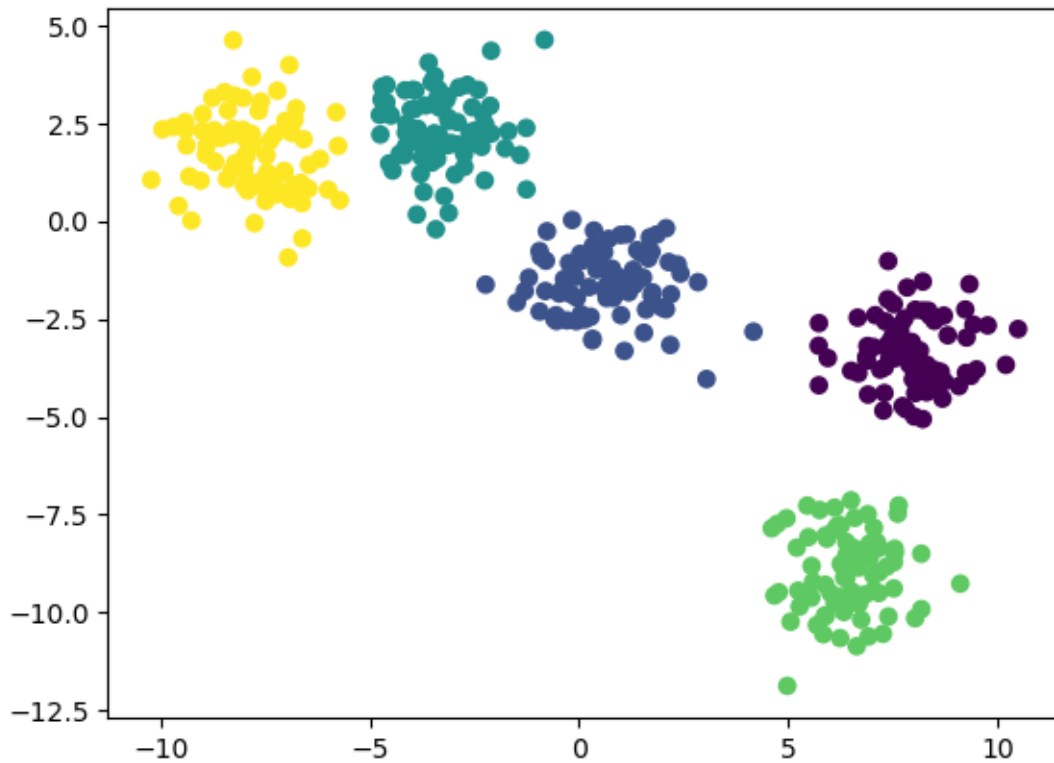


En la imagen se pueden ver claramente 5 clusters. Vamos a usar k-means para separar el conjunto de datos de manera automática.

Usando la K correcta

Primero veremos cómo se dividen los datos usando una k **correcta** de 5. Para hacer la partición se usa el método `fit_predict` que recibe los datos. El cluster correspondiente se guarda en la variable `y_km`. Luego graficamos otra vez los puntos y los coloreamos de acuerdo a su número de cluster.

```
K = 5
km = KMeans(K)
y_km = km.fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=y_km)
plt.show()
```

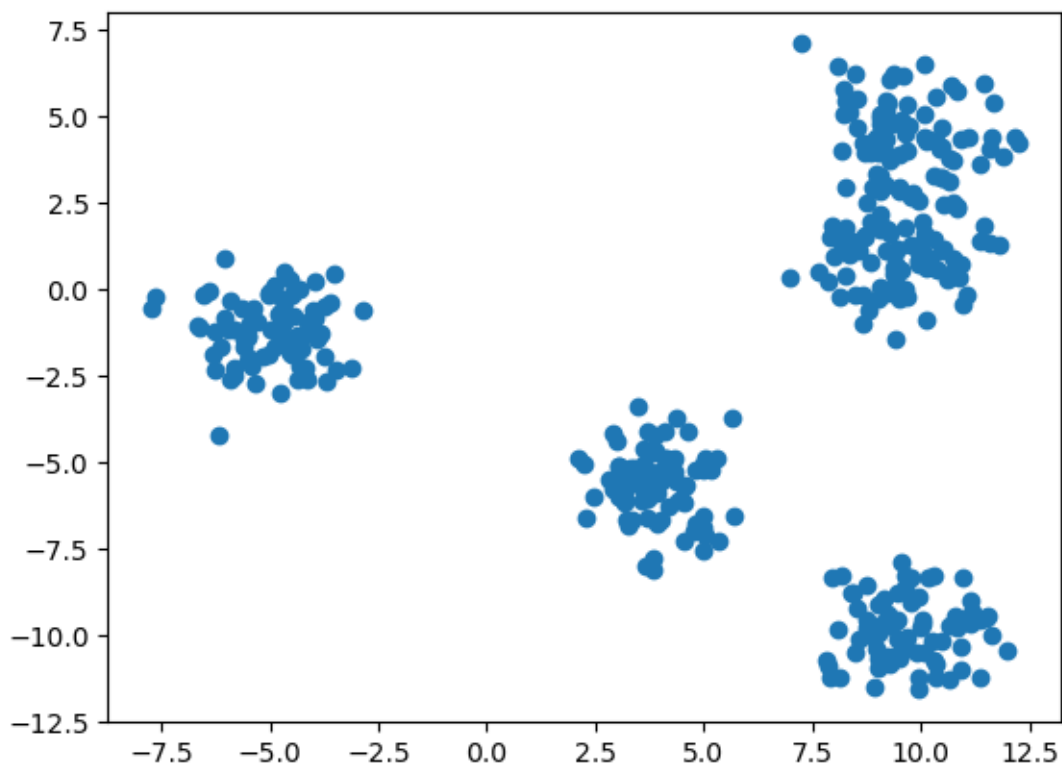


Podemos ver que al usar la k correcta, se crean los clusters de la manera esperada.

Con otra K

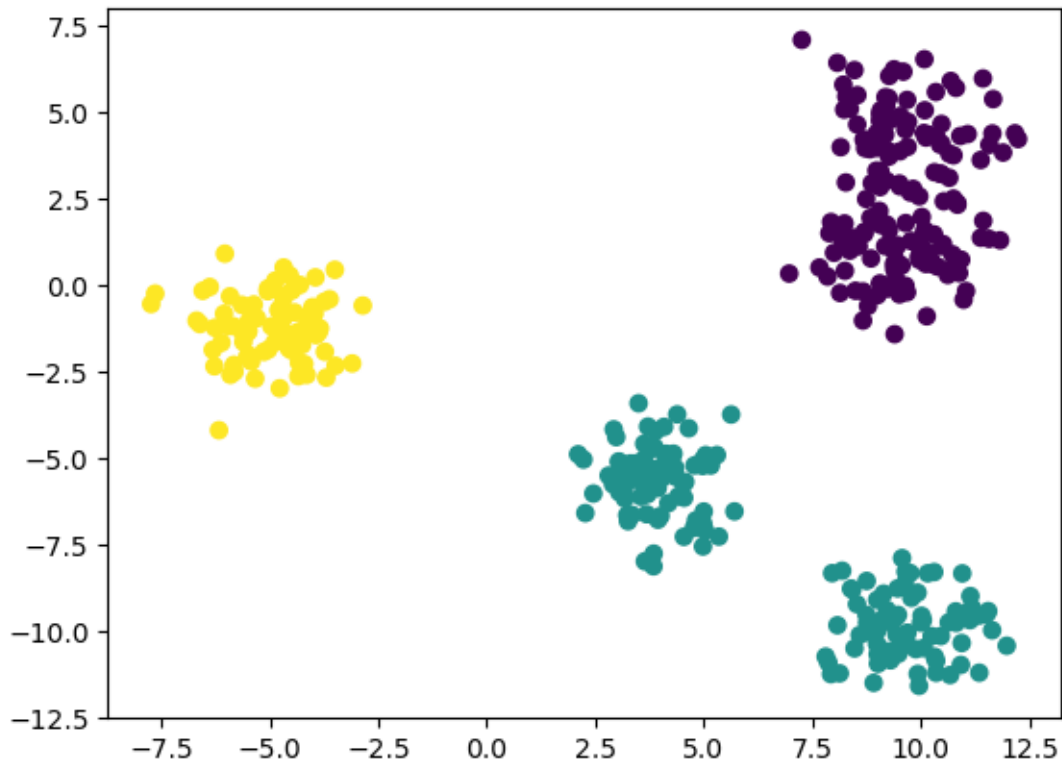
Ahora vamos a elegir varios valores de k y ver cómo se van creando los clusters. Recuerde que en principio no sabemos el valor de la k adecuado para un conjunto de datos en general.

```
X, y = make_blobs(400, 2, centers=5, random_state=4)
plt.scatter(X[:, 0], X[:, 1])
plt.show()
```



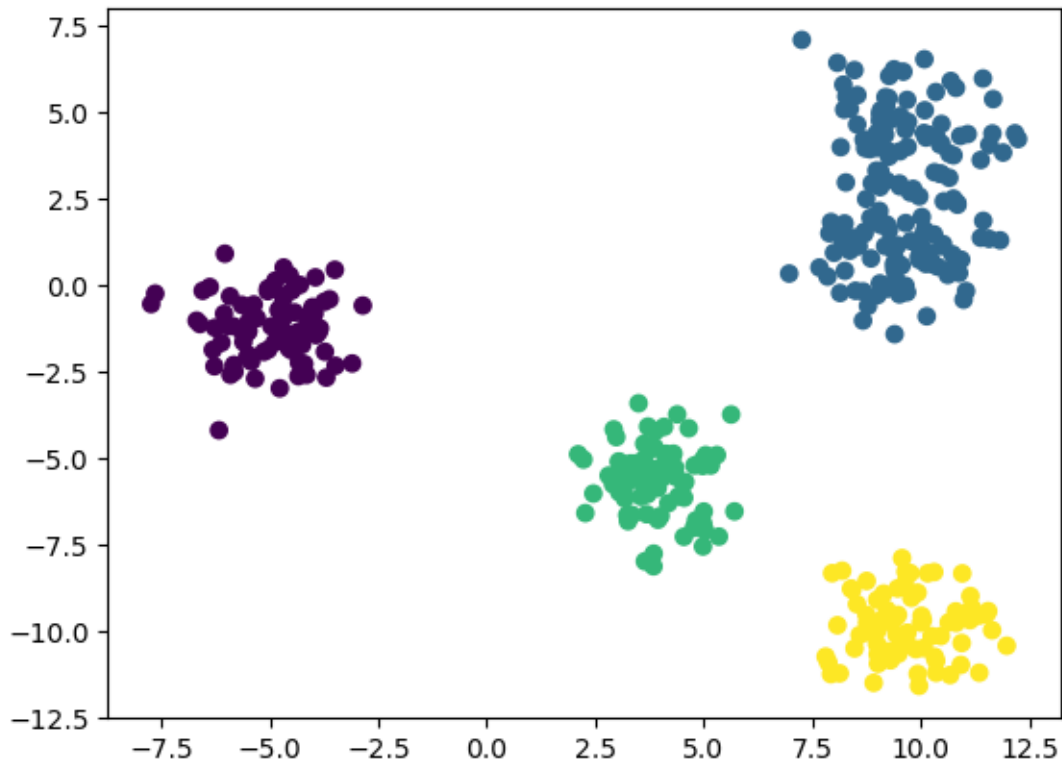
Cuando la k es pequeña, k-means pega los clusters más cercanos.

```
K = 3
km = KMeans(K)
y_km = km.fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=y_km)
plt.show()
```



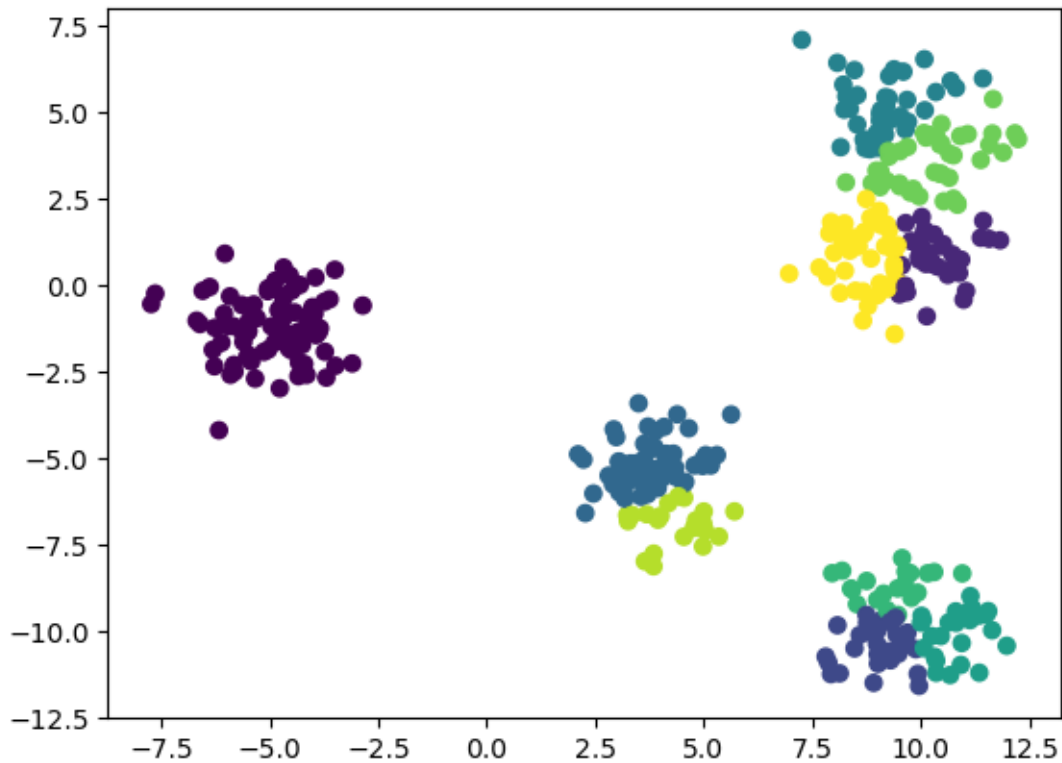
Para $k = 4$ lo que hace el algoritmo es pegar los dos clusters de la esquina que son lo más cercanos.

```
K = 4
km = KMeans(K)
y_km = km.fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=y_km)
plt.show()
```



Podemos seguir aumentando el valor de k y ver cómo cambian las particiones. En este caso vamos a probar con un valor grande, $k = 10$. Note que lo que pasa es que los clusters naturales de los datos se dividen para cubrir la cuota de 10 clusters.

```
K = 10
km = KMeans(K)
y_km = km.fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=y_km)
plt.show()
```



Actividad: Variar el conjunto de datos y comparar los resultados.

La variable `cluster_centers_` guarda los centroides de cada cluster. Podemos agregarlos a las gráficas para ver cómo se distribuyeron.

```
centers = km.cluster_centers_  
plt.scatter(X[:, 0], X[:, 1], c=y_km)  
plt.scatter(centers[:, 0], centers[:, 1], marker='x', c='r')  
plt.show()
```

